Quarry Lux Search Applet: Product Requirements

Punt Labs

August 2026

Contents

Chapter 1

Product Requirements

1.1 Overview

Quarry today has one graphical entry point: `quarry-menubar`, a native macOS `MenuBarExtra` app that connects to `quarryd` over its resolved connection (local TLS or a configured remote server) and lets a user search and read indexed content. This document specifies a second entry point: a lux applet, reachable from the shared lux menu bar whenever `quarryd` and `luxd` are both running, offering the same core search-and-read experience to any lux user regardless of platform or whether `quarry-menubar` is installed.

This is not a replacement for `quarry-menubar` and does not reduce its scope. It is an additional surface for the same backend, matching how `voxd`'s music player already gives vox a lux-native control surface alongside vox's own CLI and MCP tools.

The applet is machine-global, not session-scoped: it is composed directly into `quarryd`'s own process at daemon startup (the same shape as `voxd`'s `MusicPlayerSubsystem`), not spawned per Claude Code session. This is a product decision, not an implementation detail — a search across every registered collection, and the health of the daemon serving them, are both facts about the machine, not about any one editor session, and the applet's availability should track the daemon's uptime, not a session's.

1.1.1 Priorities and phasing

[P1] is a working search-and-read applet, at feature parity with `quarry-menubar`'s core loop (query, browse results, read full content) reachable from the lux menu bar, plus a read-only status view. [P2] is polish and parity items that improve the experience without being required to call the feature done: richer status detail, performance tuning for large result sets, and any lux-side affordance `quarry-menubar` has that this document's [P1] scope omits.

1.1.2 Goals

- Let any lux user search every collection quarry has indexed, from the lux menu bar, without installing or running `quarry-menubar`.
- Make reading a matched document's full content — not just the search snippet — the primary experience, matching what `quarry-menubar` already proved is the right shape (§UC-2, §UC-3).
- Surface enough daemon health at a glance that a user can tell whether a failed or empty search is quarry's fault or their query's fault, without leaving lux to run `quarry doctor`.

- Reuse the *contract* quarry's existing HTTP API already defines (`/v1/search`, `/v1/show`, `/v1/collections`, `/v1/status`) — the same request shapes, the same response fields — as-is where it already serves the need; extending the contract is in scope only where a requirement below cannot be met without it. **Revised (Chapter 2, §2.4):** this applet reaches that contract's underlying service layer *in-process*, not over HTTP loopback, since it runs inside `quarryd` itself rather than as an external client — the goal is contract parity with the HTTP API, not literally issuing HTTP requests to reach it.

1.1.3 Non-Goals

- Writing to quarry: no ingest, no `remember`, no delete, no collection management. This applet reads; it does not mutate quarry's index. (Distinguish from the companion control-panel applet, which does mutate — but only this repo's own capture configuration, never the index.)
- Any daemon lifecycle control (restart, service management) or connection/remote-config management (API keys, TLS trust). That stays `quarry-menubar`'s job; see § 1.1.3 below.
- Replicating `quarry-menubar`'s keyboard-first, hotkey-summoned UX exactly. Lux's own menu-bar interaction model (a click raises a frame) is the entry point here; matching `quarry-menubar`'s specific hotkey behavior is out of scope.
- Any table/filter/type-combo chrome in the style of the beads board (`apps/beads_board.py`'s live-chrome `RenderTableRequest`). Considered and explicitly deferred to a later iteration once the content-first [P1] scope ships and is validated.

Why daemon control stays out. A prior draft of this feature combined search, status, *and* daemon control (restart, database switching, sync-now, remote-serving toggles) into one applet. That was corrected: `quarry-menubar` already owns daemon-level status and control in its native surface, and duplicating that control surface in lux would create two places a user could issue conflicting daemon commands from, with no obvious source of truth. This applet's status section is read-only by design.

1.1.4 Personas

The lux-first user Runs `luxd` as their primary always-on surface across several Punt Labs tools (vox's music player, biff's presence) and wants quarry search available the same way, without a second app to install or a dock icon to manage. Primary persona for [P1].

The cross-platform user Uses lux on a platform `quarry-menubar` does not target (`quarry-menubar` is macOS-only `MenuBarExtra`; lux's Display renderer is not platform-restricted the same way). This applet is their only graphical way to search quarry.

The quarry-menubar user, occasionally Has `quarry-menubar` installed and generally prefers it, but is at a machine or in a context where lux is already open and quarry-menubar is not running; reaches for the lux applet as a fallback rather than launching a second app for one query.

Evidence status. No usage data or interviews back the cross-platform and occasional-user personas above — `quarry-menubar` already exists and plausibly serves the primary user base today, so this document ships on an assertion, not a validated need. This is treated explicitly as a validation bet, not a settled fact: § 1.5 defines what would confirm or falsify it, and [P1] is scoped to be cheap enough to ship and observe before committing to [P2].

1.2 Use Cases

Each use case names the actor, the trigger, the expected flow, and what "done well" looks like. Functional requirements in §1.3 cite the use cases they satisfy.

UC-1 Open the applet from the lux menu. [p1] *Actor: any lux user.* `quarryd` is running; the user clicks the "Quarry" entry in the lux menu bar. *Done well:* the applet's frame raises immediately (DES-063's raise-first principle) — the user sees a search field within roughly the same latency lux-beads already demonstrates for its own click-to-visible-response, not a spinner while the daemon is contacted for the first time.

UC-2 Search and scan results. [p1] *Actor: any lux user.* The user types a query into the search field. *Done well:* results appear grouped in a way that lets the user scan quickly (matching `quarry-menubar`'s grouping by source format), each row compact enough to be a picker, not a destination — the user should feel like they are choosing which document to read, not already reading it.

UC-3 Read a result's full content. [p1] *Actor: any lux user.* The user selects a result row. *Done well:* the applet shows the *full page* the result came from (via the same backing call as `quarry-menubar`'s `/v1/show`), not merely the search snippet already visible in the list — reading the content is the point of the feature, and the list exists only to get the user here quickly. Code content is legible (a monospaced or otherwise code-appropriate rendering), matching the intent of `quarry-menubar`'s syntax highlighting even if lux's rendering primitives cannot reproduce it exactly.

UC-4 Return to the list from a result. [p1] *Actor: any lux user.* The user is reading a result's content and wants to see the results list again — to pick a different result, or refine the query. *Done well:* one action returns to the list without losing the search query or re-running the search.

UC-5 Copy content out of the applet. [p1] *Actor: any lux user.* The user is reading a result and wants to paste its content elsewhere. *Done well:* the full content just read is copyable in one action, matching `quarry-menubar`'s "Copy" affordance.

UC-6 Search returns nothing. [p1] *Actor: any lux user.* The query matches no indexed content. *Done well:* the applet says so plainly (no results for this query) rather than showing an empty list indistinguishable from "still loading" or a silent failure.

UC-7 Search while the daemon is unreachable. [p1] *Actor: any lux user.* `quarryd` is down, unreachable, or the applet's own connection to it has faulted. *Done well:* the applet says the daemon is unreachable, distinctly from "no results," so the user knows to check quarry's own health rather than assume their query was bad. (See also **UC-9**.)

UC-8 Reconnect after a lux restart. [p1] *Actor: any lux user.* `luxd` restarts (or the applet's own connection drops and recovers) while `quarryd` keeps running. *Done well:* the applet's menu entry and any held content reappear automatically once the connection is back, matching the register-fresh behavior `voxd`'s music player already demonstrates on Hub reconnect — the user does not have to notice the outage or take any recovery action.

UC-9 Check daemon status without searching. [p1] *Actor: any lux user.* The user opens the applet specifically to check whether quarry is healthy — daemon up, index healthy, collections in sync — without intending to search anything. *Done well:* a status view is reachable from the same applet without a separate program or menu entry, and it is read-only: nothing in this view lets the user change daemon state (see § 1.1.3).

UC-10 Filter to one collection. [p2] *Actor: any lux user.* The user wants results from only

one indexed collection, not every collection quarry knows about — matching `quarry-menubar`'s collection picker. *Done well:* the filter is visible and changeable from the search view itself, not a separate settings screen, and persists for the remainder of the applet session (until lux or quarryd restarts).

UC-11 Scroll or page through many results. [p2] *Actor: any lux user.* A query matches more results than fit on screen at once. *Done well:* the user can reach every result, not just the first page's worth, without the applet becoming sluggish as the result count grows.

1.3 Functional Requirements

Requirements are grouped by capability area. Each cites the use case(s) it exists to satisfy and carries the same [P1]/[P2] phase tag.

1.3.1 Applet lifecycle

FR-1 [p1] The applet shall be available as a single menu entry in the lux menu bar whenever `quarryd` is running, with no dependency on any Claude Code session being open. (**UC-1**)

FR-2 [p1] Clicking the applet's menu entry shall raise its frame if it is already open, and open it with a visible search field if it is not, before any query has been issued. (**UC-1**)

FR-3 [p1] The applet shall re-register its menu entry and restore whatever it was last showing automatically on every fresh connection to `luxd` (including after a `luxd` restart), with no user action required. (**UC-8**)

1.3.2 Search

FR-4 [p1] The applet shall let the user enter a free-text query and issue a search against quarry's indexed content. (**UC-2**)

FR-5 [p1] Search results shall be grouped in a way that supports quick scanning (at minimum, by source format, matching `quarry-menubar`'s existing grouping), with each row compact enough to show document identity and a short excerpt, not full content. (**UC-2**)

FR-6 [p1] A query that matches nothing shall produce an explicit "no results" state, distinguishable from a loading state and from a daemon-unreachable state. (**UC-6**)

FR-7 [p2] The user shall be able to restrict search to one collection, selectable from the search view. (**UC-10**)

FR-8 [p2] The applet shall remain responsive as the result count grows past what fits on one screen (pagination, virtualized scrolling, or an equivalent), without a hard cap that silently drops results below the fold. (**UC-11**)

1.3.3 Reading content

FR-9 [p1] Selecting a result shall fetch and display the *full page* that result came from, not only the excerpt already visible in the results list. (**UC-3**)

FR-10 [p1] The content view shall render code-format results legibly (at minimum, a monospaced presentation that preserves whitespace and line breaks). (**UC-3**)

FR-11 [p1] One action shall return from the content view to the results list without re-issuing the search or losing the query text. (**UC-4**)

FR-12 [p1] One action shall copy the full content currently displayed. (**UC-5**)

1.3.4 Daemon status (read-only)

FR-13 **[p1]** The applet shall offer a status view, reachable from the same applet, showing at minimum: daemon reachability, and whether the last search attempt succeeded or failed for a daemon-side reason. (**UC-9**)

FR-14 **[p1]** When the daemon is unreachable, the applet shall say so explicitly and distinctly from a genuine zero-result search, both in the search view and the status view. (**UC-7**)

FR-15 **[p1]** The status view shall contain no control that changes daemon state (no restart, no database switch, no configuration write). (**UC-9**; see § 1.1.3)

FR-16 **[p2]** The status view shall list the daemon's known collections by name and document/chunk counts, the data `/v1/status` already exposes today (`QuarryModels.swift`'s `StatusResponse`: aggregate `document_count`, `collection_count`, `chunk_count` — no per-collection breakdown). A per-collection last-synced indication, if wanted, is a genuinely new field this requirement does not assume exists; scoping it is an open question (§ 5), not something `quarry-menubar` already provides. (**UC-9**)

1.4 Non-Functional Requirements

NFR-1 **[p1]** Menu-click to first visible content (a search field, or a held prior result) shall not be materially worse than lux-beads' own demonstrated click-to-visible-response latency (DES-063 measured lux-beads at roughly 55 ms median for a raise, with the slow `bd` read moving behind that raise, not blocking it). The equivalent principle applies here: raise first, fill behind it.

NFR-2 **[p1]** A daemon fault (crash, restart, or a fatal fault in either lux leg) shall not require the applet's host process (`quarryd`) to be restarted by a human — the subsystem shall self-heal, matching `MusicPlayerSubsystem`'s guarded restart-with-backoff loop.

NFR-3 **[p2]** The applet shall not measurably degrade `quarryd`'s own search latency for other clients (the HTTP API, `quarry-menubar`, the MCP server) merely by being connected and idle.

1.5 Success Metric

This applet exists on an unvalidated persona bet (§ 1.1 personas). The falsifiable claim: within the first month after [P1] ships, at least some non-trivial share of searches happen through this applet from users who also have `quarry-menubar` available — evidence that the applet is reaching a real, distinct moment of use (lux already open, quarry-menubar not running) rather than sitting unused beside an app that already does the job. If usage never materializes among users who could have used `quarry-menubar` instead, [P2] (collection filtering, pagination, richer status) is not justified and this applet's further investment should be reconsidered before, not after, building it.

1.6 Open Questions

1. **Resolved in Chapter 2.** Whether lux's board/scene primitive supports replacing a board's content in place is settled: yes, by pushing a fresh `RenderRequest` to the same `frame_id` the results table already used — no new board plumbing needed. See Chapter 2, § 2.2 for the three-part proof.

2. What quarry HTTP API surface, if any, is missing for this applet versus what `quarry-menubar` already calls (`/v1/search`, `/v1/documents`, `/v1/collections`, `/v1/show`, `/v1/status`)?
3. Does `quarryd` taking a direct dependency on `punt_lux` (as `voxd` does) have any packaging consequence for quarry, which ships as a CLI+MCP hybrid plugin today?
4. **Still open; carried forward as Chapter 2, §2.5 Decision 4.** What, precisely, should **FR-13**'s status view show beyond reachability — is FD headroom, FTS index health, and sync recency (all already in `quarry doctor`'s output) the right read-only subset, or is that too much detail for a glance-and-close use case?
5. **FR-16** originally assumed `quarry-menubar`'s status surface already exposes a per-collection last-synced timestamp; peer review found it does not — `/v1/status` returns only aggregate counts (`StatusResponse` in `QuarryModels.swift`). Is a per-collection last-synced field worth adding to quarry's HTTP API for this applet, and if so is `quarry doctor`'s existing sync machinery (`compute_sync_plan`) the right source for it, or is aggregate-only status sufficient for [P1]?

Chapter 2

Technical Design Solution

This chapter resolves the open questions raised in Chapter 1 against the actual code of the three repositories this feature spans (lux, vox, quarry). Nothing here is speculative where the cited source settles the question; where it does not, the gap is named as a decision required before implementation, not silently assumed.

2.1 Component overview

Contingent on Decision 1 (§2.5). Everything in this section assumes `quarryd` takes `punt_lux` as a direct, mandatory dependency, mirroring `voxd`'s own precedent. That decision is not yet ratified (§2.5, Decision 1) — if it is ruled the other way, this component shape does not apply and needs re-deriving against whatever constraint motivated the different ruling.

The applet is a new `QuarrySearchSubsystem`, living in `src/quarry/daemon/`, composed into `quarryd` at startup and run as a fire-and-forget task for the daemon's lifetime — the same shape as `MusicPlayerSubsystem` in `voxd` (`vox/src/punt_vox/voxd/music_player/composition.py`), imported directly and unconditionally in `voxd/daemon.py:38`, not behind a feature flag or optional import. It owns two self-healing legs under one `asyncio.TaskGroup`, restarting both together on any fatal fault with backoff, exactly as `MusicPlayerSubsystem.run()` does:

The push leg Owns a `LuxClient` built for `quarryd`'s own app identity (`LuxClient.for_identity`, per `lux/src/punt_lux/client/facade.py:76-78`) and pushes scene content — the search results table, the detail view, and the status view — to one named scene/frame. Modeled on `LuxScenePublisher`.

The receive leg Subscribes to the events this applet's UI can raise (a query submission, a result selection, a tab switch between Search and Status) and, on each, does the corresponding work and re-pushes. Modeled on `LuxSubscription`/`VoxPanelLeg`'s `on_event` dispatch (`vox/src/punt_vox/panel/leg.py:128-130`).

Both legs share one `on_connect` hook that registers the menu entry and re-pushes whatever the subsystem last held, firing on every fresh Hub handshake — this is what satisfies **FR-3** (re-registration after a `luxd` restart) for free, since it is the same mechanism `MusicPlayerSubsystem` already uses for the identical requirement in vox.

2.2 Resolved: the list/detail swap (Open Question 1)

Resolved: no new board plumbing is needed. Reading `lux/src/punt_lux/applets/board_load.py` and `board_glass.py` in full settles this. Three facts, together:

1. `BeadsBoard.request()`, called from `BoardLoad.board()` (`board_load.py:73-75`), already returns `RenderTableRequest | RenderRequest` for *the same* `scene_id`/`frame_id` in the existing beads applet — a table for issues, a plain scene for a failure or empty message. The Hub already composes heterogeneous request shapes pushed to one frame; this is not new behavior this applet would be the first to need.
2. `RenderRequest.elements` (`lux/src/punt_lux/operations/models/render.py:58`) is `list[dict[str, object]]` — an open, self-validating element tree, not a short-message-only type. A full page's content, scrolled and rendered as text elements, is exactly the kind of thing this type already exists to carry.
3. `BoardGlass.shows()` serializes pushes under one lock and always re-reads the held state *inside* that lock before writing, so two pushes racing (a stale list-refresh landing after a fresh detail-view push, or vice versa) cannot land out of order — the newer one always wins, by construction, not by a place counter that could itself race.

The design, therefore: one scene id (e.g. `quarry-search`) serves both the results list and the detail view. Selecting a result pushes a fresh `RenderRequest` of the full page's content to that same scene id, replacing the table in place; returning to the list (**FR-11**) pushes the held `RenderTableRequest` back. `BoardSlot` and `BoardGlass` are reusable directly — neither is session-applet-specific, and neither's construction depends on the `AppletProgram`/`SessionClaim` composition this applet does not use. `AppletBoard` is *not* reusable as-is: it is transitively bound to the beads domain through its concrete `@final BoardLoad` dependency. See "Decisions required," below — this is a real, currently-unscoped, cross-repo piece of work, not a free reuse.

2.3 Search and selection events

Unlike `lux-beads`, whose click carries no payload (a click just means "reload," `beads.py` takes no arguments beyond session identity), this applet's clicks need to carry data: a query string, and a selected result's identity (document, page, collection, chunk index) sufficient to call `/v1/show`. `VoxPanelService`'s `apply_event(topic, payload)` (`vox/src/punt_vox/panel/service.py:146-174`) is the precedent: a payload-carrying event on a named topic, dispatched to the field it changes, each topic named by an enum (`PanelTopic`) rather than a bare string, so a typo in a topic name fails at import time, not at runtime.

Two topics, minimum: `QUERY` (payload: the query string) and `SELECT_RESULT` (payload: enough of a result's identity to re-fetch it). Both ride the same `listener().subscribe(*topics)` call the push/receive legs already share, per `VoxPanelLeg._listen_once` (`leg.py:96-105`).

2.4 Reaching quarry's own search

`QuarrySearchSubsystem` runs *inside* `quarryd`'s own process. It must call the same service layer the `/v1/search` and `/v1/show` route handlers call — `SearchService` in `src/quarry/daemon/routes/search.p` and its counterpart in `routes/show.py` (`route_table.py` only wires route specs to these handler methods; the handler bodies, and the service calls worth reusing, live in `routes/`) — in-process, never looping back through its own HTTP API over loopback TLS to reach code already running in the same process. This is a decision, not an oversight worth flagging as an open question: the HTTP layer's job is authenticating and shaping requests for external clients; a subsystem composed into the daemon itself is not an external client and gains nothing from paying that overhead.

2.5 Decisions required before implementation

1. **Packaging: `punt-lux` as a direct, mandatory dependency.** `voxd` imports `MusicPlayerSubsystem` directly and unconditionally (`daemon.py:38`), and `punt-lux>=0.28.0` is a plain dependency in `vox/pyproject.toml`, not an optional extra. The precedent is a hard dependency, not a feature-flagged one — `quarryd` would take the same shape: `punt_lux` always importable, the subsystem always composed, tolerating `luxd` being absent or down *at runtime* (both `MusicPlayerSubsystem` and `VoxPanelLeg` retry forever rather than crash their host on a connection failure) but not absent from the dependency graph at install time. This needs a ratified decision before implementation, since it changes quarry's install footprint for every user, not only those who run lux.

2. **`AppletBoard` is not actually reusable as-is; this needs a lux-side change.** Peer review found §2.2's "reuse `AppletBoard`/`BoardSlot`/`BoardGlass` directly" is imprecise. `AppletBoard.__new__(cls, load: BoardLoad, slot: BoardSlot)` (`applet_board.py:43`) requires a concrete `BoardLoad` instance, and `BoardLoad` is itself `@final` (`board_load.py:40`), constructed from concrete, beads-domain `BeadsBoard`/`BeadsSource` objects (`punt_lux.apps.beads_boar` `AppletBoard` is transitively bound to the beads domain through `BoardLoad` today. `BoardSlot` and `BoardGlass` genuinely have no such binding and are reusable as claimed; only `AppletBoard` is not. Reusing it for quarry requires a lux-side change — genericizing `BoardLoad` (a Protocol, or a type parameter) so a `QuarrySearchBoardLoad` can satisfy `AppletBoard`'s constructor — which is cross-repo work this document cannot unilaterally scope. Confirm with lux's own maintainers before assuming this reuse path is free.

3. **App-identity client factory.** `voxd` injects its `LuxClient` construction behind `VoxLuxClients` (a factory Protocol, swapped for a fake in tests) rather than calling `LuxClient.for_identity` inline. Confirm quarry's daemon testing conventions (`tests/hermetic_env.py`'s embedding-fake pattern) extend cleanly to a second client boundary, or whether this needs its own hermeticity mechanism.

4. **Status-view data source.** Per the corrected **FR-16** (Chapter 1), per-collection last-synced data does not exist in quarry's HTTP API today. Deciding whether to add it (and from what source — `doctor.py`'s `compute_sync_plan`, or a new field) is a real scoping decision for whoever picks up this design, not resolved here.

5. **Status-view scope beyond reachability, carried forward from Chapter 1.** Chapter 1's open questions ask, and this chapter does not resolve, exactly what **FR-13** should show beyond daemon reachability — FD headroom, FTS index health, and sync recency are all already in `quarry doctor`'s output and are candidates, but including all three may be more detail than a glance-and-close status view warrants. Genuinely undecided; needs a ruling before **FR-13** can be implemented as more than a bare up/down indicator.

6. **Success-metric instrumentation.** §1.6's success metric (search volume through this applet, among users who also have `quarry-menubar` available) has no instrumentation point designed anywhere in this chapter. As currently specified, nothing in `QuarrySearchSubsystem` records enough to answer the metric's question after [P1] ships. Before implementation, either design a minimal usage-logging mechanism (even a simple query-count-with-timestamp is likely sufficient) or explicitly accept the metric will need to be assessed some other way (e.g. a direct ask to users), and say so.

2.6 Known unknowns

Board sizing and layout for a scrollable full-page detail view have not been measured against lux's ImGui element set for a large document (a multi-thousand-line source file, for instance) —

whether long-content rendering performs acceptably, or needs its own pagination/virtualization the way **FR-8** already anticipates for the results list, is untested and should be spiked before [P1] is considered feature-complete, not assumed from the type system alone.